

An Analysis of CycleGAN

CycleGAN

CycleGAN -- Part 1

In practice The generative network generates candidates while the discriminative network evaluates them. This creates a contest based on data distributions, where the generator learns to map from a latent space to the true data distribution, aiming to produce candidates that the discriminator cannot distinguish from real data. The discriminator's goal is to correctly identify these candidates, but as the generator improves, its task becomes more challenging, increasing the discriminator's error rate. A known dataset serves as the initial training data for the discriminator. Training involves presenting it with samples from the training dataset until it achieves acceptable accuracy. The generator is trained based on whether it succeeds in fooling the discriminator. Typically, the generator is seeded with randomized input that is sampled from a predefined latent space (e.g. a multivariate normal distribution). Thereafter, candidates synthesized by the generator are evaluated by the discriminator. Independent backpropagation procedures are applied to both networks so that the generator produces better samples, while the discriminator becomes more skilled at flagging synthetic samples. When used for image generation, the generator is typically a deconvolutional neural network, and the discriminator is a convolutional neural network.

CycleGAN -- Part 2

Conditional GAN Conditional GANs are similar to standard GANs except they allow the model to conditionally generate samples based on additional information. For example, if we want to generate a cat face given a dog picture, we could use a conditional GAN. The generator in a GAN game generates μ_G , a probability distribution on the probability space Ω . This leads to the idea of a conditional GAN, where instead of generating one probability distribution on Ω , the generator generates a different probability distribution $\mu_G(c)$ on Ω , for each given class label c . For example, for generating images that look like ImageNet, the generator should be able to generate a picture of cat when given the class label "cat". In the original paper, the authors noted that GAN can be trivially extended to conditional GAN by providing the labels to both the generator and the discriminator. Concretely, the conditional GAN game is just the GAN game with class labels provided: $L(\mu_G, D) := E_{c \sim \mu_C, x \sim \mu_{ref}(c)} [\ln D(x, c)] + E_{c \sim \mu_C, x \sim \mu_G(c)} [\ln (1 - D(x, c))]$ where μ_C

μ_C is a probability distribution over classes, $\mu_{ref}(c)$ is the probability distribution of real images of class c , and $\mu_G(c)$ the probability distribution of images generated by the generator when given class label c . In 2017, a conditional GAN learned to generate 1000 image classes of ImageNet.

CycleGAN -- Part 3

Two time-scale update rule The two time-scale update rule (TTUR) is proposed to make GAN convergence more stable by making the learning rate of the generator lower than that of the discriminator. They prove that when trained this way, GANs "converge, under mild assumptions to a stationary local Nash equilibrium". They further show that this property extends to the use of the Adam optimizer, which is commonly used in stochastic gradient descent. It is important to note, however, that a local Nash equilibrium in no way signifies an absence of mode collapse - for instance, a GAN trained on MNIST collapsed to generating a single digit may satisfy the hypotheses of the paper, while still presenting mode collapse.

CycleGAN -- Part 4

Knight, Will. "5 Big Predictions for Artificial Intelligence in 2017". MIT Technology Review. Retrieved January 5, 2017. Karras, Tero; Laine, Samuli; Aila, Timo (2018). "A Style-Based Generator Architecture for Generative Adversarial Networks". arXiv:1812.04948 [cs.NE]. This Person Does Not Exist – photorealistic images of people who do not exist, generated by StyleGAN This Cat Does Not Exist Archived March 5, 2019, at the Wayback Machine – photorealistic images of cats who do not exist, generated by StyleGAN Wang, Zhengwei; She, Qi; Ward, Tomas E. (2019). "Generative Adversarial Networks in Computer Vision: A Survey and Taxonomy". arXiv:1906.01529 [cs.LG].

CycleGAN -- Part 5

StyleGAN-2 StyleGAN-2 improves upon StyleGAN-1, by using the style latent vector to transform the convolution layer's weights instead, thus solving the "blob" problem. This was updated by the StyleGAN-2-ADA ("ADA" stands for "adaptive"), which uses invertible data augmentation as described above. It also tunes the amount of data augmentation applied by starting at zero, and gradually increasing it until an "overfitting heuristic" reaches a target level, thus the name "adaptive".

CycleGAN -- Part 6

An Analysis of CycleGAN

CycleGAN

$$L(G) = \mathbb{E}_{x \sim \mu_G} [(D(x) - c)^2]$$
where a, b, c are parameters to be chosen. The authors recommended $a = -1, b = 1, c = 0$.

CycleGAN -- Part 7

Choice of the strategy set In the most generic version of the GAN game described above, the strategy set for the discriminator contains all Markov kernels $\mu_D : \Omega \rightarrow \mathcal{P}[0, 1]$, and the strategy set for the generator contains arbitrary probability distributions μ_G on Ω . However, as shown below, the optimal discriminator strategy against any μ_G is deterministic, so there is no loss of generality in restricting the discriminator's strategies to deterministic functions $D : \Omega \rightarrow [0, 1]$. In most applications, D is a deep neural network function. As for the generator, while μ_G could theoretically be any computable probability distribution, in practice, it is usually implemented as a pushforward: $\mu_G = \mu_Z \circ G^{-1}$. That is, start with a random variable $z \sim \mu_Z$, where μ_Z is a probability distribution that is easy to compute (such as the uniform distribution, or the Gaussian distribution), then define a function $G : \Omega_Z \rightarrow \Omega$. Then the distribution μ_G is the distribution of $G(z)$. Consequently, the generator's strategy is usually defined as just G , leaving $z \sim \mu_Z$ implicit. In this formalism, the GAN game objective is
$$L(G, D) := \mathbb{E}_{x \sim \mu_{\text{ref}}} [\ln D(x)] + \mathbb{E}_{z \sim \mu_Z} [\ln (1 - D(G(z)))]$$